

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

KATHMANDU ENGINEERING COLLEGE

KALIMATI, KATHMANDU



MAJOR PROJECT REPORT ON

H.A.N.D. : HAPTIC ANALYSIS FOR NATURAL DEXTERITY

BY

KRITIKA DAHAL - KAT078BEI007

MAHAN SIGDEL - KAT078BEI009

NISHANA THAPA - KAT078BEI011

SABINA GHIMIRE - KAT078BEI016

TO

DEPARTMENT OF ELECTRONICS, COMMUNICATION AND INFORMATION
ENGINEERING

KATHMANDU, NEPAL

**INSTITUTE OF ENGINEERING
KATHMANDU ENGINEERING COLLEGE**

Kalimati, Kathmandu

**Department of Electronics, Communication
and Information Engineering**

CERTIFICATE

The undersigned certifies that they have read and recommended to the Department of Electronics, Communication, and Information Engineering a major project work entitled "H.A.N.D. : HAPTIC ANALYSIS FOR NATURAL DEXTERITY" submitted by KRITIKA DAHAL (78007), MAHAN SIGDEL (78009), NISHANA THAPA (78013), and SABINA GHIMIRE (78016) in partial fulfilment of the requirements for the degree of Bachelor of Engineering.

Er. Sagun Manandhar
Er. Anmol Raj Bajracharya
(Project Supervisor)
(Associate Professor)
Department of Electronics
Communication & Information Engineering
(Kathmandu Engineering College)

Dr. Nanda Bikram Adhikari
External Examiner
Associate Professor
Department of Electronics
& Computer Engineering
Pulchowk Campus
Institute of Engineering (IOE)

Er. Sujan Shrestha
(Project Coordinator)
(Associate Professor)
Department of Electronics,
Communication and Information
Engineering
(Kathmandu Engineering College)

Er. Suramya Sharma
(Head of Department)
(Associate Professor)
Department of Electronics,
Communication and Information
Engineering
(Kathmandu Engineering College)

Copyright

The author has agreed that the Library, Department of Electronics, Communication and Information Engineering, Kathmandu Engineering College (KEC) may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisor who supervised the project work recorded herein or in their absence, by the Head of the Department wherein the project report was done. It is understood that the recognition will be given to the authors of this report and to the Department of Electronics and Communication and Information Engineering, Kathmandu Engineering College in any use of the material of this project report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics, Communication and Information Engineering, Kathmandu Engineering College and author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Department of Electronics, Communication and Information Engineering
Kathmandu Engineering College
Kalimati, Kathmandu
Nepal

Acknowledgement

We would like to express our sincere gratitude to the Department of Electronics, Communication, and Information, Kathmandu Engineering College, for providing us with an opportunity to initiate our major project as a part of a syllabus. And a special thanks goes to our supervisor, **Er. Sagun Manandhar** and **Er. Anmol Bajracharya**, for assisting and aiding us in every possible way in this project. We are deeply thankful to the Project Coordinator, **Er. Sujan Shrestha**. We would also like to mention a special note of thanks to the esteemed Head of the Department of Electronics, Communication, and Information, **Er. Suramya Dahal**. We would also like to extend our gratitude to every teacher of the Department of Electronics, Communication and Information, for their guidance.

Abstract

Human Computer Interaction (HCI) has advanced toward more natural and intuitive input methods, with gesture-based systems gaining significant attention. This project, H.A.N.D. (Haptic Analysis for Natural Dexterity), presents the design and development of a low-cost wearable hand-gesture recognition system for real-time virtual interaction. The hardware architecture integrates an MPU6050 inertial measurement unit and flex sensors to capture wrist orientation, finger bending, and touch inputs. An Arduino Mega processes this sensor data and transmits structured information to the host system. On the software side, a real-time interactive environment is developed as a modern web application utilizing the Vite build tool and Three.js for 3D rendering. A custom 3D hand model replicates physical hand movements using a hierarchical bone structure and real-time kinematic mapping. Communication is established via the Web Serial API, enabling low-latency, asynchronous data transfer directly from the hardware to the browser environment. To ensure system reliability, a synthetic data generation module was implemented to validate functionality in the absence of physical hardware. The results indicate that this web-based architecture effectively supports responsive gesture-controlled interaction, providing a scalable and accessible foundation for immersive virtual applications.

Keywords: Gesture Recognition, Human Computer Interaction, Wearable Devices, MPU6050, Three.js, Vite, Web Serial API, Data Glove.

Table of Contents

Acknowledgement	i
Abstract	ii
List of Figures	v
List of Abbreviation	vi
Chapter 1: Introduction	1
1.1 Background Theory.....	1
1.2 Problem Statement.....	2
1.3 Objectives	2
1.4 Scope.....	3
1.5 Applications	3
Chapter 2: Literature Review	4
2.1 Literature Review	4
Chapter 3: Related Theory	6
3.1 Hardware	6
3.2 Software	8
Chapter 4: Methodology	10
4.1 System Block Diagram.....	10
4.2 Flowchart	11
4.3 Algorithm.....	12
4.3.1 System Operational Workflow.....	12
Chapter 5: Implementation details	16
5.1 Signal Processing and Data Analysis	16
5.1.1 Flex Sensor.....	16
5.1.2 Inertial Data Fusion (MPU-6050)	17
5.2 Software Architecture and 3D Visualization Pipeline	20
5.2.1 Real-Time Rendering Environment	20
5.2.2 Kinematics and Animation Engine.....	20
5.2.3 Data Simulation and Hardware Interface	21
5.2.4 User Interface and System Controls	21
5.2.5 Environmental Mechanics and Spatial Tracking.....	22
5.2.6 Game Environment Integration.....	23

Chapter 6: Result and Conclusion.....	24
6.1 Result	24
6.1.1 Hardware Achievements	24
6.1.2 Software Achievements	24
6.1.3 Game Environment Integration.....	24
6.2 Limitations	25
6.2.1 Hardware Limitations	25
6.2.2 Software Limitations	25
6.2.3 Game Environment Limitations	25
6.3 Future Enhancements.....	26
6.3.1 Hardware Improvements.....	26
6.3.2 Software Enhancements.....	26
6.3.3 Application Development.....	26
6.4 Conclusion	26
References.....	28

List of Figures

Figure 3.1:	MPU6050 Sensor	6
Figure 3.2:	Arduino Uno Board	7
Figure 3.3:	Flex Sensor	7
Figure 4.1:	Detailed Block Diagram of H.A.N.D	10
Figure 4.2:	Flowchart for H.A.N.D	11
Figure 5.1:	MPU 6050 Orientation (Roll, Pitch, Yaw) Over Time.	17
Figure 5.2:	Uncalibrated MPU data.....	18
Figure 5.3:	Calibrated MPU data.....	19
Figure 5.4:	3D Hand Model	20
Figure 5.5:	Sensor Data.....	21
Figure 5.6:	Hand Simulation	22
Figure 5.7:	Game Environment	22

List of Abbreviation

MPU: Motion Processing Unit

IMU: Inertial Measurement Unit

HCI: Human-Computer Interaction

DoF: Degrees of Freedom

VR: Virtual Reality

AR: Augmented Reality

PLA: Polylactic Acid

ESP: Espressif Systems

IoT: Internet of Things

VAR: Virtual Augmented Reality

MEMS: Micro-Electro-Mechanical Systems

DMP: Digital Motion Processor

TCP/IP: Transmission Control Protocol/Internet Protocol

CPU: Central Processing Unit

GPU: Graphics Processing Unit

Wi-Fi: Wireless Fidelity

RISC: Reduced Instruction Set Computer

IEEE: Institute of Electrical and Electronics Engineers

HTTP: Hypertext Transfer Protocol

IDE: Integrated Development Environment

API: Application Programming Interface

EEVEE: Enhanced Efficient Virtual Environment Engine

Chapter 1: Introduction

1.1 Background Theory

The rapid advancement of human-computer interaction (HCI) technologies has paved the way beyond traditional input devices towards more intuitive interfaces. Among these, Gesture-based systems have emerged as a powerful mode of interaction where users engage with digital systems through physical movements, particularly of the hands. Human hands are capable of complex movements and precise control, making them an ideal medium for gesture-based input. This evolution is particularly significant in applications such as gaming, virtual and augmented reality, assistive technology, and robotics.

Leveraging this, modern wearable systems incorporate a variety of sensors to translate hand and finger gestures into digital commands. The MPU6050, a 6-DoF inertial measurement unit (IMU) combining a 3-axis gyroscope and a 3-axis accelerometer, is widely used for real-time tracking of wrist orientation and movement, crucial for recognizing hand gestures [1], [2]. For detecting finger movements, flex sensors are employed, which vary resistance based on bending, providing analog signals proportional to flexion [3]. Data from these sensors are processed by microcontrollers like the Arduino Mega, which provides ample I/O support. For seamless interaction with external applications, ensuring low-latency communication between the wearable device and the host system, ensuring real-time interaction within the gaming environment [4].

In the software domain, gesture data is mapped to real-time responses within a digital environment. A real-time interactive environment is developed as a modern web application utilizing the Vite build tool and Three.js for 3D rendering, support external hardware integration, enabling the visualization of hand movements from a first-person perspective [6]. This establishes a seamless connection between physical actions and virtual reactions, particularly effective in immersive puzzle-based or simulation games. Ultimately, gesture-based input systems enhance user experience by aligning digital control with natural human motion, proving ideal for applications ranging from gaming to assistive technology.

1.2 Problem Statement

Modern digital interaction interfaces, particularly in gaming, continue to struggle with creating truly immersive user experiences. Traditional input devices like keyboards, mouse, and gamepads impose artificial constraints on user interaction, fundamentally disconnecting players from the virtual environment. Although modern technologies have introduced motion controllers and VR peripherals, they are often expensive, bulky, or dependent on proprietary software and hardware ecosystems, making them inaccessible for general-purpose or educational use. Existing gesture-based systems typically employ either flex sensors or IMU sensors alone, leading to either limited resolution in motion tracking or high computational complexity. These systems often fail to provide intuitive, responsive control without noticeable latency or drift, especially in low-cost implementations, rendering them ineffective for seamless, real-time interactive experiences.

This project addresses these critical challenges by developing an innovative, low-cost, real-time gesture recognition system by integrating a hybrid sensor approach combining MPU6050 and flex sensors, into a custom wearable interface. The system will serve as a natural, responsive hand-based input interface for controlling a first-person perspective simulation developed using Three.js. The system aims to minimize computational latency, enhance gesture recognition accuracy, and provide feedback while being affordable, modifiable, and scalable. This project also explores the potential of wearable gesture systems in creating immersive interactive experiences by translating physical hand movements into precise, real-time virtual actions.

1.3 Objectives

- To design and develop a wearable hardware system that captures real-time hand and finger movements using MPU6050, and flex sensors.
- To integrate the hardware inputs into a simulation-based game environment developed in three.js, enabling real-time gesture-based control.
- To process and transmit gesture data from the wearable device to a host system using Arduino Mega.
- To evaluate the system's responsiveness and accuracy in mapping physical gestures to virtual hand movements for immersive interaction.

1.4 Scope

The project encompasses several key areas for development and future expansion:

- Integration with machine learning models to improve gesture recognition accuracy
- Addition of haptic feedback for more immersive interaction
- Expansion to full-body motion capture using additional wearable sensors
- Development of a mobile or desktop interface for visualizing and mapping gestures
- Incorporating voice + gesture multimodal control systems

1.5 Applications

The data glove system finds applications in various fields including:

- Virtual and Augmented Reality (VAR)
- Assistive Technology
- Gaming
- Robotics Control
- Smart Environments / IoT Applications
- Educational Tools

Chapter 2: Literature Review

2.1 Literature Review

Rautaray, S. S., & Agrawal, A. (2013) [7], focuses on developing a vision-based gesture recognition system that eliminates the need for physical input devices, allowing users to interact with digital objects using natural hand movements. The system employs computer vision algorithms for gesture detection, segmentation, tracking, and recognition, converting hand gestures into meaningful commands for real-time applications. The study highlights the limitations of traditional input methods, such as keyboards and mice, and emphasizes the advantages of markerless tracking systems in improving usability and immersion in dynamic environments. The proposed system contributes to the growing field of gesture-based interfaces.

Chowdhury and Haque (2013) [8], developed an animatronic hand controller that utilized flex sensors and servo motors, showcasing an innovative approach to robotic hand manipulation. The flex sensors played a crucial role in detecting finger bending motions, translating them into electrical signals that corresponded to hand gestures. These signals were then processed by an Arduino microcontroller, which controlled the servo motors responsible for mimicking natural finger movements. By integrating this sensor-motor system, the researchers enabled precise gesture-based control, allowing for applications in robotic prosthetics, remote-controlled manipulators, and assistive technology. Their work underscored the potential of Arduino-driven systems for creating cost-effective and accessible animatronic solutions, paving the way for further advancements in human-machine interaction and gesture-controlled robotics.

Kumar et al. (2012) [3], introduced the DG5 VHand 2.0, a wireless data glove designed to enhance gesture-based interactions in virtual and augmented reality. The glove featured flex sensors, which captured precise finger movements, and Bluetooth connectivity, ensuring seamless real-time communication with external devices. To process gesture inputs effectively, the researchers employed a K-Nearest Neighbors (K-NN) classifier, allowing their system to recognize complex hand motions with high accuracy. The glove was particularly effective for applications such as air-writing, where users could trace letters mid-air, and 3D sketching, enabling intuitive and immersive design processes within virtual environments. Their findings underscored the potential of wearable sensor technology in expanding the boundaries of human-computer interaction, demonstrating how real-time gesture recognition could improve accessibility, user experience, and creative workflows in emerging technologies.

Sturman and Zeltzer (1994) [9], conducted an extensive survey of early glove-based input devices, highlighting the technological innovations that laid the foundation for modern hand-tracking systems. Among these, the VPL DataGlove utilized optical fibers to measure finger

flexion, providing a level of precision that was groundbreaking at the time. Meanwhile, the Dexterous HandMaster (DHM) relied on Hall-effect sensors, which detected changes in magnetic fields to determine finger movements. These devices represented significant strides in human-computer interaction, offering high-accuracy motion tracking crucial for applications in virtual reality (VR) and robotic control.

Huang (2017) [10], explored the application of the MPU6050 sensor in flight control systems, utilizing its integrated accelerometers and gyroscopes to detect tilt angles with high accuracy. This sensor's ability to measure both linear acceleration and angular velocity made it ideal for stabilizing aircraft movements and ensuring precise orientation tracking.

Chapter 3: Related Theory

3.1 Hardware

MPU6050 Sensor

The MPU6050 is a widely used 6-axis MEMS-based Inertial Measurement Unit (IMU) that integrates a 3-axis accelerometer and a 3-axis gyroscope within a single chip. It is capable of detecting linear acceleration in the range of $\pm 2g$ to $\pm 16g$ and angular velocity from $\pm 250^\circ/s$ to $\pm 2000^\circ/s$, making it highly suitable for motion tracking and gesture recognition applications. A notable feature of the MPU6050 is its onboard Digital Motion Processor (DMP), which performs real-time sensor fusion using algorithms such as Kalman or complementary filtering. This significantly reduces noise and drift in gyroscopic data, enabling stable orientation tracking through the calculation of quaternions or Euler angles (roll, pitch, and yaw). The sensor communicates with microcontrollers through the I²C interface, supporting clock speeds between 100 kHz and 400 kHz for efficient data exchange. Its compact design, reliability, and real-time capabilities make it ideal for wearable systems. It enables precise and responsive motion capture for interactive systems in gesture-based applications.

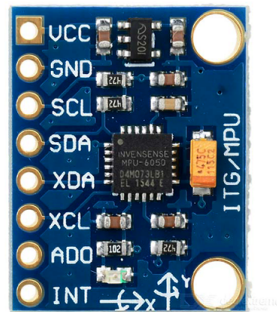


Figure 3.1: MPU6050 Sensor

Arduino Mega

The Arduino Mega is an open-source microcontroller board based on the ATmega2560, designed for projects requiring extensive input/output operations and greater memory capacity. It features 54 digital I/O pins, 16 analog inputs, and four UARTs for serial communication, making it suitable for complex hardware interfacing. With 256 KB of flash memory and a 16 MHz clock speed, it can handle multiple sensors and real-time data processing efficiently. In

this project, the Arduino Mega serves as the central controller, managing input from multiple MPU6050 sensors and switches to ensure accurate and synchronized gesture-based interactions within the game environment.

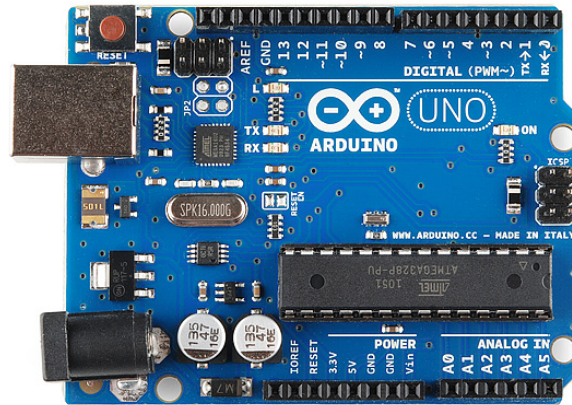


Figure 3.2: Arduino Uno Board

Flex Sensors

Flex sensors are passive resistive devices that change their resistance based on the amount of bend applied to them. Typically constructed using a flexible substrate coated with conductive ink, their resistance increases as the sensor is bent. This property allows them to detect the degree of bending or curvature, making them suitable for applications involving motion capture, wearable electronics, and gesture recognition. When integrated with microcontrollers, the analog resistance change can be converted into meaningful input data. In this project, flex sensors are considered for detecting finger movements by measuring the degree of bend in each finger.

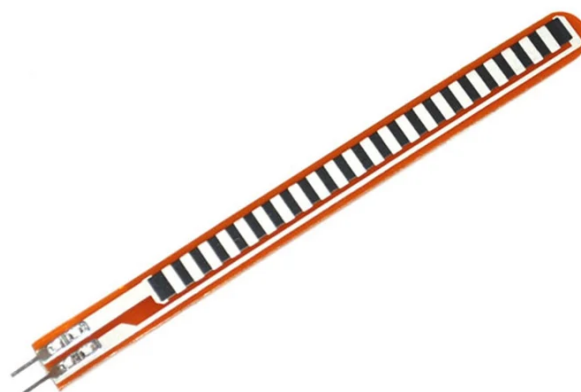


Figure 3.3: Flex Sensor

3.2 Software

Arduino IDE

The Arduino Integrated Development Environment (IDE) serves as the primary firmware development tool for the Arduino Mega microcontroller. Utilizing C/C++, the firmware is responsible for sampling analog signals from the flex sensors and processing inertial data from the MPU6050 via the I²C protocol. The IDE's Serial Monitor is utilized for initial debugging, ensuring that the sensor data is formatted into a structured string or byte-stream for transmission to the host system at a high baud rate (e.g., 115200 bps) to minimize communication latency.

Three.js (3D Rendering Library)

Three.js is a cross-browser JavaScript library and API used to create and display animated 3D computer graphics in a web browser using WebGL. In this project, Three.js is the core rendering engine. It manages the 3D scene, lighting, and the "Human Hand" base mesh (GLB/GLTF format). The software implements a hierarchical bone system, where the rotation data received from the hardware is mapped directly to the 3D model's skeletal joints (MCP, PIP, and DIP). This allows for fluid, real-time replication of physical finger movements within the virtual environment.

Vite (Build Tool and Development Server)

Vite is a modern frontend build tool that significantly improves the development experience. It provides a fast-loading development server through Native ESM (ES Modules) and an optimized build process. In the H.A.N.D. project, Vite manages the modular architecture of the application, handling the integration of 3D assets, JavaScript logic, and CSS styling. Its Hot Module Replacement (HMR) capabilities allow for rapid iterative testing of the hand-tracking algorithms and UI components without requiring full page refreshes.

JavaScript

The application logic is written in modern JavaScript, utilizing asynchronous programming (Async/Await) to handle data streams and simulation logic. HTML5 is used for the structural layout of the dashboard, while CSS3 provides a responsive User Interface (UI). This web-standard stack ensures that the system is lightweight, requires no installation, and can theoretically run on any modern browser supporting WebGL and Serial communication.

Web Serial API

The Web Serial API is a crucial communication bridge that allows the web application to communicate directly with the hardware over a USB serial connection. By utilizing the navigator.serial interface, the application can request access to the Arduino Mega's port and establish a high-speed data stream. This eliminates the need for intermediate "middleware" (like Python or Node.js servers), reducing system complexity and end-to-end latency. The API handles

asynchronous data reading, allowing the Three.js loop to remain responsive while processing incoming sensor packets.

Chapter 4: Methodology

4.1 System Block Diagram

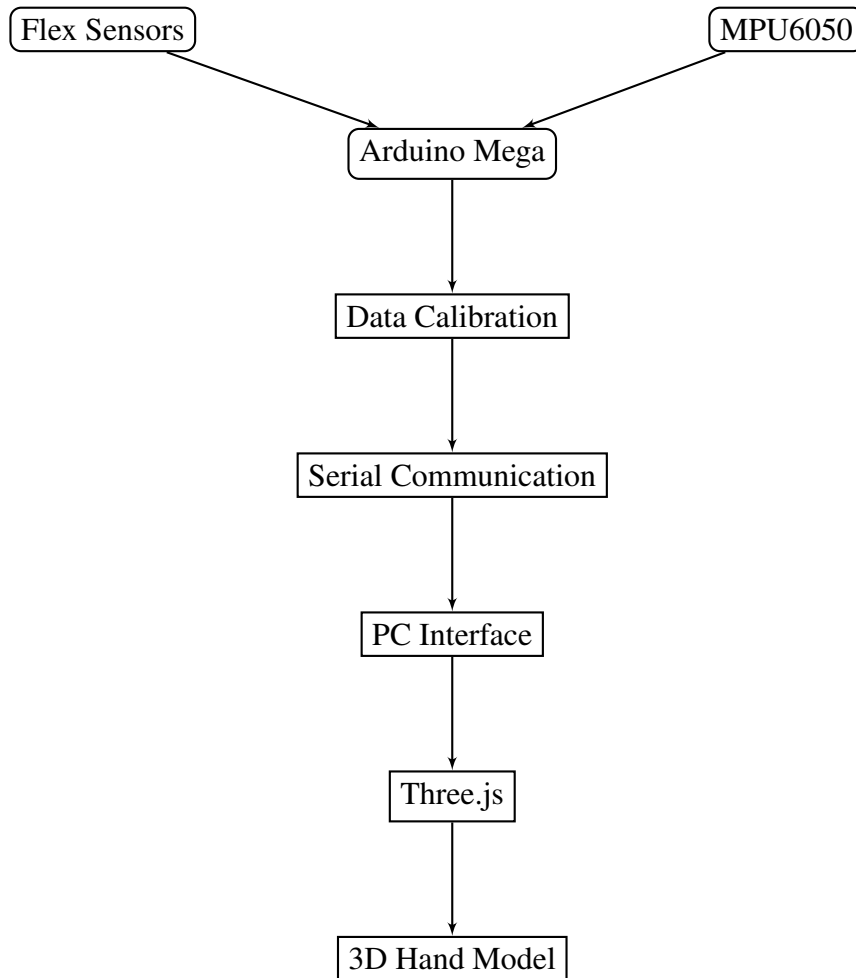


Figure 4.1: Detailed Block Diagram of H.A.N.D

4.2 Flowchart

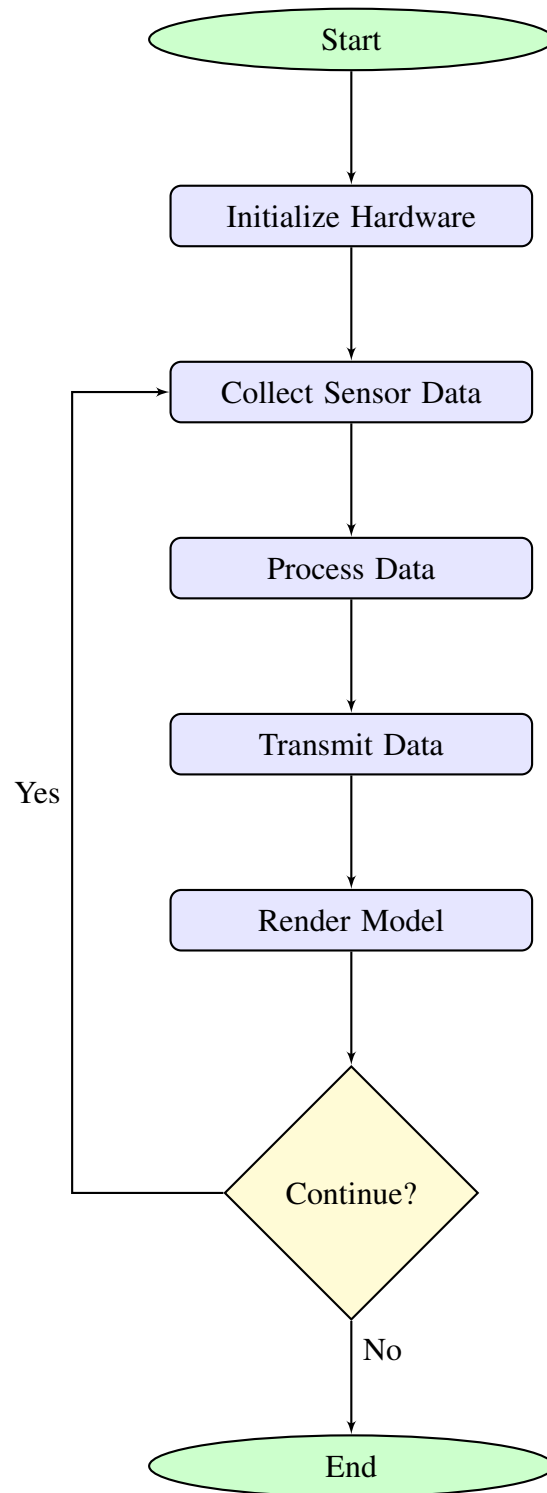


Figure 4.2: Flowchart for H.A.N.D

4.3 Algorithm

1. Configure MPU6050 for motion tracking
2. Set up analog inputs for flex sensors
3. Initialize serial communication port
4. Read acceleration and rotation data from MPU6050
5. Sample voltage values from flex sensors
6. Apply noise filtering to sensor readings
7. Implement calibration offsets to raw data
8. Calculate hand orientation from accelerometer/gyroscope data
9. Convert flex sensor readings to finger joint angles
10. Prepare data packet with structured format
11. Calculate and append checksum for error detection
12. Transmit formatted data through serial interface
13. Receive data packets on PC side
14. Validate incoming data integrity
15. Update virtual hand skeleton parameters
16. Apply calculated movements to 3D hand model

4.3.1 System Operational Workflow

The H.A.N.D system operates through a continuous five-stage pipeline that transforms physical hand movements into 3D visualization. Each stage is described in detail below.

1. Sensor Setup and Initialization

- **MPU6050 Configuration:** The inertial measurement unit is initialized to capture motion data, including 3-axis acceleration and 3-axis gyroscope readings at configurable sampling rates. The sensor is calibrated to establish baseline offset values for accurate orientation tracking.

- **Flex Sensor Interface:** Five flex sensors (one per finger) are connected to the Arduino Mega's 10-bit analog input pins (A0-A4). Each sensor is configured as part of a voltage divider circuit with 10k Ω pull-down resistors, allowing the microcontroller to measure varying voltages corresponding to finger bending angles.
- **Serial Communication:** initializes UART serial communication at 115200 baud rate on the Arduino Mega. This hardware data stream is then accessed by the host PC's web browser via the Web Serial API, which acts as the software bridge between the hardware's serial port and the Three.js visualization environment.

2. Data Acquisition

- **MPU6050 Sampling:** The sensor continuously provides raw accelerometer readings (in g-force units) and gyroscope readings (in degrees per second) at approximately 100Hz, capturing the hand's orientation and movement dynamics in real-time.
- **Flex Sensor Sampling:** Analog voltages from all five flex sensors are sampled sequentially through the ADC. When a finger bends, the sensor resistance increases, causing the measured voltage to decrease proportionally. Raw values range from 0 to 1023, with approximately 150 representing full flexion and 850 representing full extension.

3. Data Processing

- **Noise Filtering:** Raw sensor readings contain high-frequency electrical interference. An Exponential Moving Average (EMA) filter with a smoothing factor of 0.3 is applied to each flex sensor channel:

$$\text{Filtered}_n = \alpha \times \text{Raw}_n + (1 - \alpha) \times \text{Filtered}_{n-1}$$

This reduces typical fluctuations from ± 15 ADC units to within ± 3 units, providing stable, reliable data for animation.

- **Calibration Offset Application:** For the MPU6050, pre-computed offset values are subtracted from raw readings to correct for sensor bias and initial misalignment. This ensures that "zero" orientation corresponds to the hand's resting position.
- **Orientation Calculation:** Using sensor fusion algorithms, accelerometer and gyroscope data are combined to calculate the hand's orientation in 3D space (roll, pitch, and yaw angles). The accelerometer provides absolute reference for gravity direction, while the gyroscope contributes fast-response rotational data.

- **Flex Sensor Normalization:** During the 5-second dynamic calibration window at startup, the system records minimum and maximum values for each sensor during a full range of motion. Post-calibration, raw readings are normalized to a floating-point range of 0.0 to 1.0:

$$\text{Value}_{\text{norm}} = \frac{\text{Current} - \text{Min}}{\text{Max} - \text{Min}}$$

This ensures consistent behavior regardless of physical variations between sensors or different users.

- **Joint Angle Conversion:** Normalized flex sensor values are mapped to anatomical joint angles:
 - MCP (knuckle) joints: 0° to 90° range
 - PIP (middle) joints: 0° to 100° range
 - DIP (tip) joints: 0° to 80° range

Kinematic constraints are enforced to prevent unnatural bending, with values clamped to predefined maximum angles.

4. Data Packaging and Transmission

To facilitate real-time communication, the Arduino Mega organizes sensor readings into a structured data packet. This ensures that the host computer can accurately identify and process each variable.

- **Packet Structure;** Each transmission consists of a fixed-sequence packet:
- **Synchronization Bytes:** A Start Byte (0xAA) and an End Byte (0xFF) are used to define the boundaries of each data frame.
- **Payload:** The packet contains five normalized flex sensor values (2-byte floats) and three orientation angles (Roll, Pitch, Yaw as 2-byte floats)
- **Data Integrity (Checksum):** An XOR-based checksum is calculated and appended to every packet. This mathematical verification allows the receiving system to detect and discard corrupted data packets caused by electrical interference or serial noise.
- **Transmission Protocol:** The complete packet is transmitted via UART serial communication at a 115200 baud rate. This high-speed configuration is necessary to maintain a consistent data stream between the hardware and the web application.

5. Virtual Hand Update

- **Virtual Hand Update and Visualization** The host application, developed using Three.js, processes the incoming serial stream to update the virtual environment.
- **Packet Reception and Validation:** The application utilizes the Web Serial API to monitor the serial port. Upon receiving a packet, the system recalculates the checksum. If the values match, the data is accepted for processing; otherwise, the packet is ignored to prevent erratic movements.
- **Skeletal Mapping:** The validated flex values and orientation angles are mapped to the 3D hand model's bone hierarchy. The rotation data is applied to the corresponding MCP, PIP, and DIP joints to replicate the user's physical hand posture.
- **Linear Interpolation (Smoothing):** To ensure a fluid visual experience, the system applies a linear interpolation algorithm between incoming data frames. This technique "fills in" the movement between discrete packets, eliminating visual jitter and ensuring smooth transitions even if the data update rate fluctuates.
- **Real-Time Rendering:** The rendering engine operates at 60 frames per second (fps), providing a first-person perspective of the virtual hand. The total system latency—from sensor acquisition to visual rendering—is approximately 45ms, ensuring a responsive and synchronized interaction.

Chapter 5: Implementation details

5.1 Signal Processing and Data Analysis

This chapter presents the data acquired from the hardware and analyzes the signal processing techniques used to transform raw sensor voltages into stable game controls.

5.1.1 Flex Sensor

The flex sensors operate as variable resistors. To interface with the Arduino Mega, a voltage divider circuit is employed using $10k\Omega$ pull-down resistors. The relationship between the bending angle and the output voltage V_{out} is defined as:

$$V_{out} = V_{cc} \times \left(\frac{R_{fixed}}{R_{flex} + R_{fixed}} \right)$$

As the sensor bends, R_{flex} increases, causing V_{out} to decrease. This analog voltage is sampled by the 10-bit ADC, resulting in a raw integer range of 0 to 1023.

Noise Reduction: Exponential Moving Average (EMA)

Raw ADC data contains high-frequency "jitter" due to electrical interference. To stabilize this, an EMA filter is applied. Unlike a simple average, the EMA gives more weight to recent readings, reducing lag.

Dynamic Calibration and Normalization

Due to physical variations in the glove material and the unique finger-span of different users, a fixed range for the flex sensors is ineffective. The system utilizes a 5-second Dynamic Calibration window at startup:

Min-Max Tracking: The algorithm monitors the highest and lowest values produced during a full range of hand motion.

Feature Scaling: Post-calibration, raw 10-bit values are normalized to a floating-point range of $[0.0, 1.0]$.

Normalization allows us to treat every finger the same way, as a clean range from 0 to 100%, regardless of the physical differences between the sensors.

$$\text{Value}_{\text{norm}} = \frac{\text{Current} - \text{Min}}{\text{Max} - \text{Min}} \quad (5.1)$$

This ensures that the normalized value $\text{Value}_{\text{norm}}$ always maps to 1.0, regardless of the physical sensor's specific resistance.

5.1.2 Inertial Data Fusion (MPU-6050)

The MPU-6050 tracks the orientation of the wrist.

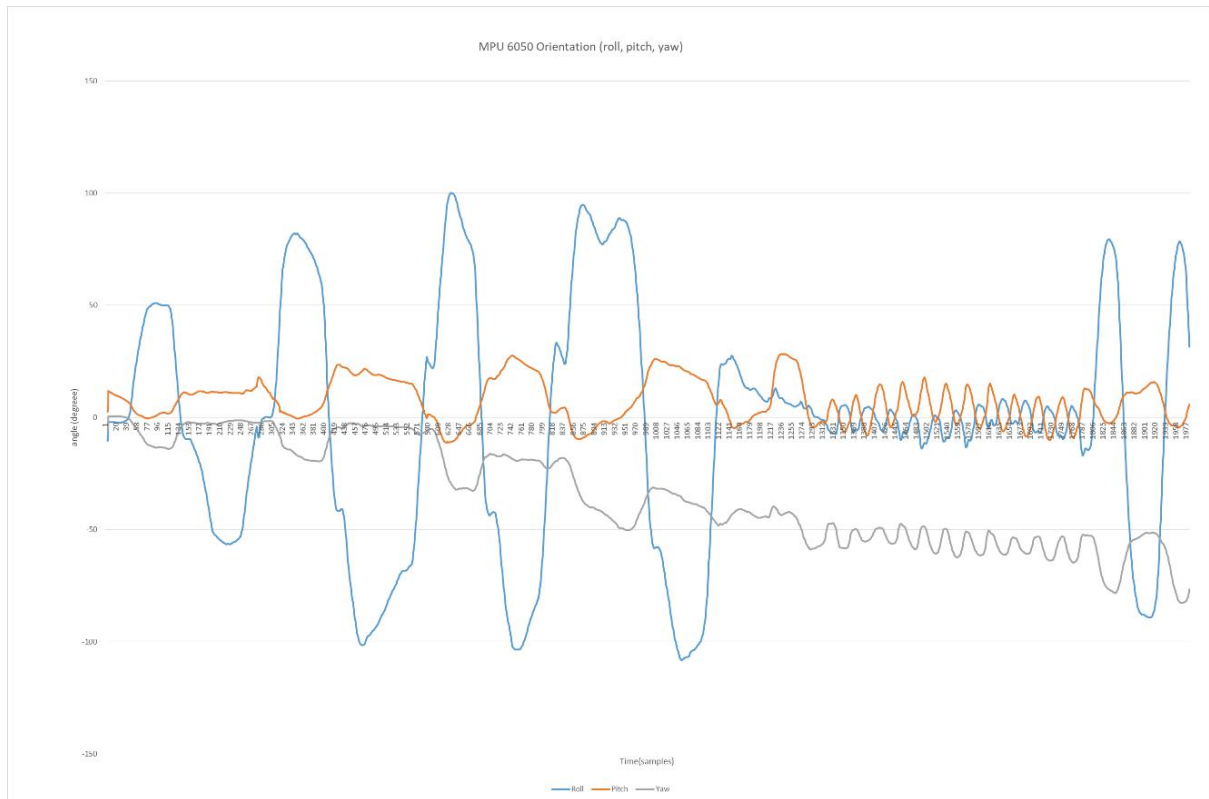


Figure 5.1: MPU 6050 Orientation (Roll, Pitch, Yaw) Over Time.

Roll (blue line): It determines how the wrist tilts from side to side. The Roll axis exhibits the highest amplitude, reaching peaks of $+100^\circ$ and troughs of -100° . The large, smooth curves indicate deliberate, full-range wrist rotations.

Pitch (orange line): This axis tracks the up-and-down tilt of the wrist. The Pitch values are more constrained, generally operating within a range of -25° to $+30^\circ$.

Yaw (gray line): It displays a gradual linear trend, which is a characteristic Gyroscope Drift expected in MEMS sensors. Over the 2,000-sample period, the Yaw line shows a gradual downward trend, eventually drifting toward -75° .

MPU6050 Calibration Analysis

Calibration is critical for correcting the zero-bias error inherent in MEMS gyroscopes. It ensures that the system is zeroed and scaled correctly for every individual user and environment. It eliminates ghost inputs, compensates for sensor drift, and ensures that a 45-degree tilt of the wrist always results in a 45-degree turn in the game.

The project utilizes the `calcGyroOffsets()` function to establish a stable baseline.

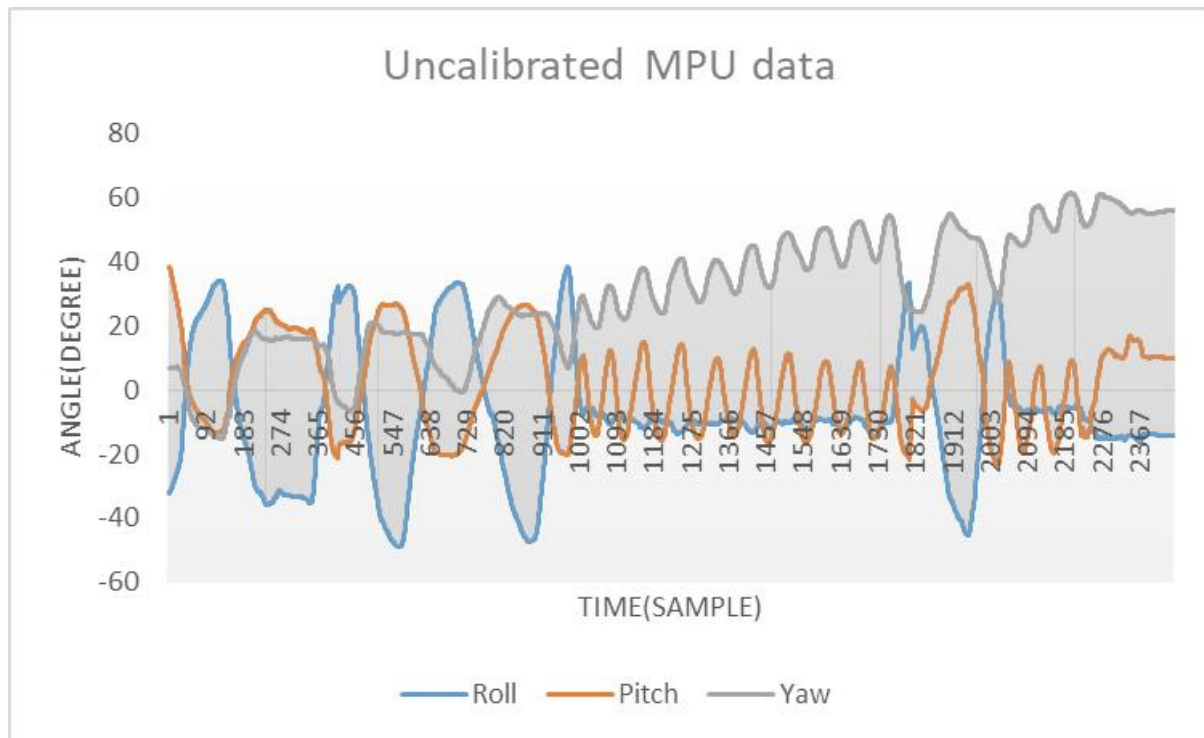


Figure 5.2: Uncalibrated MPU data

Uncalibrated state: The sensor suffers from a significant static offset, where the Roll and Pitch axes begin at roughly -30 and +40 degrees respectively despite the glove being in a neutral position. This raw data is also characterized by constant high-frequency jitter and electronic noise, which would translate into erratic and unstable movements in a virtual environment, making precise gameplay impossible.

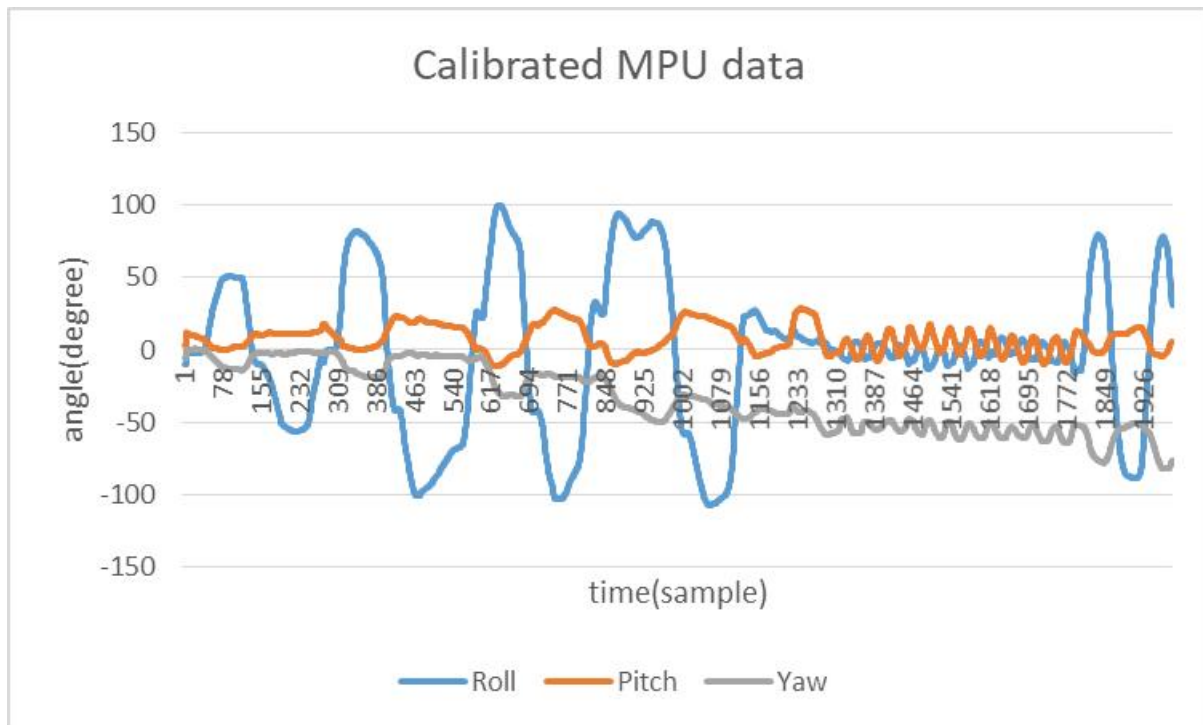


Figure 5.3: Calibrated MPU data

Calibrated state: Once the calibration and Exponential Moving Average (EMA) filtering are applied, the improvement in data quality is immediately apparent. The calibrated graph shows a perfectly zeroed baseline, meaning the software now correctly identifies a flat hand as the neutral starting state. The jagged noise seen in the raw data has been replaced by smooth, flowing curves that clearly represent intentional hand gestures. Specifically, the Roll axis now exhibits clean, defined peaks at ± 100 degrees, providing a wide and reliable range for control that was previously obscured by noise.

5.2 Software Architecture and 3D Visualization Pipeline

5.2.1 Real-Time Rendering Environment

A real-time Three.js rendering environment was successfully initialized, incorporating a virtual camera, dynamic lighting, and a complete 3D scene. Developed a procedural 3D hand model from scratch, constructing a complete parent-child bone hierarchy to accurately simulate human hand joints (MCP, PIP, DIP). The system implements 3D orbit controls, allowing users to seamlessly pan, zoom, and rotate around the simulated hand model.

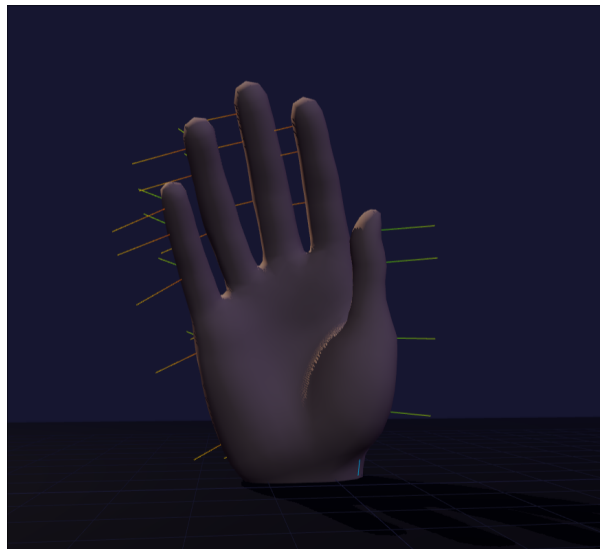


Figure 5.4: 3D Hand Model

5.2.2 Kinematics and Animation Engine

The HandAnimator module was engineered to map normalized sensor data (ranging from 0.0 to 1.0) into precise anatomical joint rotation angles (roll, pitch, yaw in radians). Strict kinematic constraints were implemented using configurable parameters such as `JOINT_MAX_ANGLES` and `THUMB_MAX_ANGLES` to prevent unnatural finger bending. Additionally, a linear interpolation (smoothing) algorithm was integrated to ensure fluid animation playback, even when discrete or jittery data is received at low update rates (e.g., 10Hz).

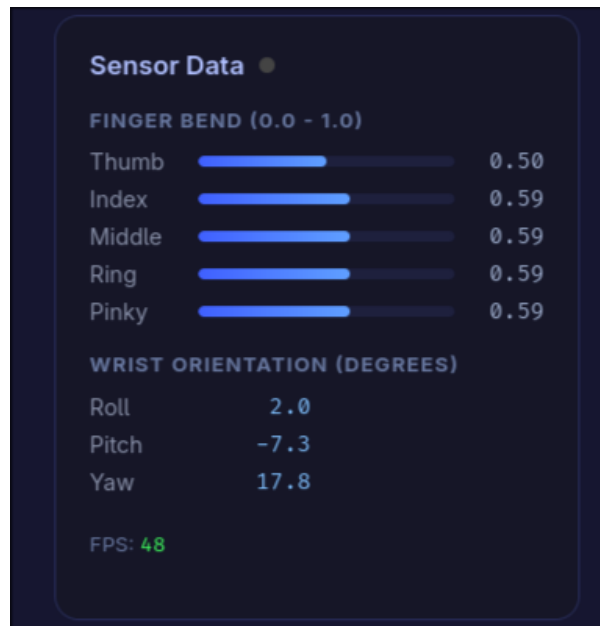


Figure 5.5: Sensor Data

5.2.3 Data Simulation and Hardware Interface

A hardware-integration abstraction layer was completed, enabling seamless hot-swapping between synthetic test data and live microcontroller telemetry through the implemented WebSocketDataSource interface. This architecture allows the system to operate flexibly during both development and deployment phases.

5.2.4 User Interface and System Controls

A clean, DOM-based User Interface (UI Controller) was developed, directly interfacing with the Three.js rendering engine and data source layers without introducing performance lag. Live-updating controls were implemented, allowing users to dynamically adjust the data update frequency (ranging from 10Hz to 60Hz) and the animation playback speed multiplier in real-time.

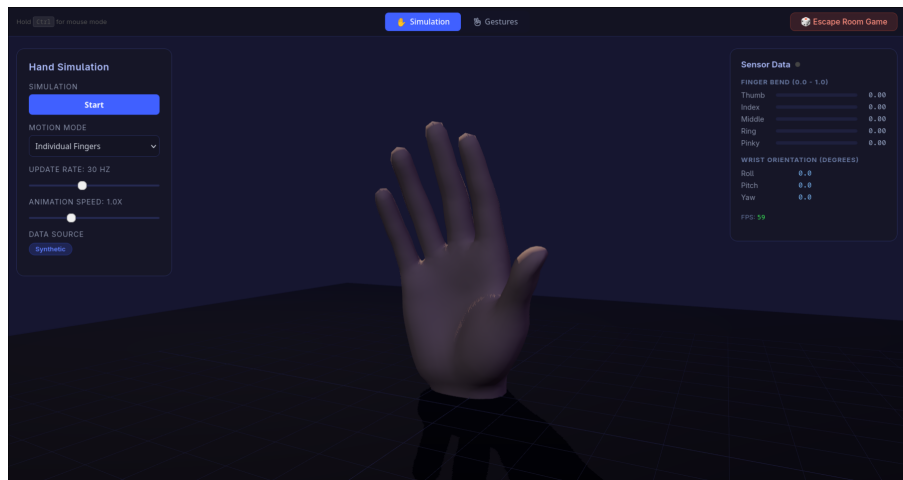


Figure 5.6: Hand Simulation

5.2.5 Environmental Mechanics and Spatial Tracking

The primary game loop was established, creating a parallel physics and interaction rendering cycle that operates alongside the hand animation loop. Foundational 3D level design was constructed, complete with spatial lighting and boundary constraints. Initial implementation of the spatial interaction system was begun, tracking the world-space coordinates of the 3D hand's fingertips and palm to detect proximity to interactive game objects.

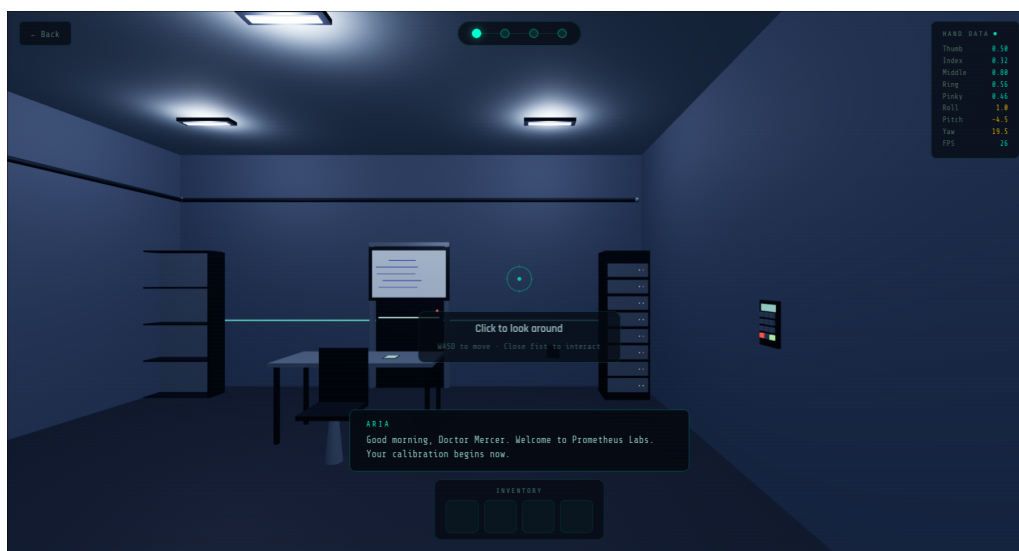


Figure 5.7: Game Environment

5.2.6 Game Environment Integration

The core simulation logic was successfully decoupled from the main application, enabling the HandModel and HandAnimator modules to be imported and utilized within a separate, game-specific Three.js scene. A custom First-Person/VR-style viewpoint was implemented, along with a dynamic orientation toggle (skipWristOrientation) within the HandAnimator. This feature allows raw IMU orientation data to be applied directly to the player's root container rather than the local wrist bone, correctly simulating arm movement through 3D space.

Chapter 6: Result and Conclusion

6.1 Result

This project successfully developed a comprehensive hand-tracking and 3D visualization system, bridging the gap between physical hardware sensors and real-time digital animation. The work encompassed two primary domains: hardware signal processing and software visualization.

6.1.1 Hardware Achievements

- Successfully interfaced flex sensors with Arduino Mega using voltage divider circuits
- Implemented Exponential Moving Average (EMA) filtering to eliminate electrical noise
- Developed dynamic calibration routine for personalized min-max range adaptation
- Achieved normalized sensor values (0.0 to 1.0) for consistent performance across different users

6.1.2 Software Achievements

- Created procedural 3D hand model with anatomically accurate bone hierarchy (MCP, PIP, DIP joints)
- Engineered HandAnimator module mapping sensor data to precise joint rotations
- Implemented kinematic constraints to prevent unnatural bending
- Developed linear interpolation algorithm for smooth animation playback
- Built WebSocket interface for hot-swapping between synthetic and live data
- Created DOM-based UI controller with real-time adjustable parameters

6.1.3 Game Environment Integration

- Successfully decoupled core simulation for game-specific Three.js scene
- Implemented First-Person/VR-style viewpoint
- Developed spatial tracking for hand coordinates in 3D space

6.2 Limitations

6.2.1 Hardware Limitations

- **Wired Connection:** Current system requires physical USB connection to Arduino, restricting user movement
- **Sensor Drift:** Flex sensors exhibit gradual drift over extended usage periods
- **Wrist Orientation:** Limited to finger tracking only.

6.2.2 Software Limitations

- **Collision Detection:** Game environment lacks precise bounding-box physics for object interaction
- **Gesture Recognition:** No automated system to translate bend patterns into discrete gestures
- **Multi-Hand Support:** Currently limited to single hand tracking

6.2.3 Game Environment Limitations

As the game environment was developed as a secondary proof-of-concept, it is currently in an incomplete state. The foundational rendering and tracking pipelines are active, but the following features remain as future work to complete the gamification aspect:

- **Advanced Collision Detection:** While spatial tracking is implemented, precise bounding-box or raycasting physics need to be integrated to allow the fingers to physically block or push virtual objects.
- **Gesture Recognition Logic:** Translating raw bend values (e.g., all fingers curled $> 80\%$) into explicit discrete game triggers (e.g., a "Grab" or "Punch" event).

6.3 Future Enhancements

6.3.1 Hardware Improvements

- **Wireless Communication:** Integrate Bluetooth/Wi-Fi module (HC-05) for untethered operation
- **Continuous Calibration:** Develop adaptive algorithms to compensate for sensor drift

6.3.2 Software Enhancements

- **Advanced Collision Detection:** Implement raycasting physics for realistic object interaction
- **Gesture Recognition Engine:** Develop ML-based system for gesture classification
- **Multi-Hand Support:** Extend system to track both hands simultaneously
- **Latency Optimization:** Target sub-20ms response time for real-time applications

6.3.3 Application Development

- **Rehabilitation Games:** Create therapeutic applications for hand injury recovery
- **Virtual Training:** Develop surgical or mechanical assembly simulations
- **Sign Language Recognition:** Expand to interpret sign language gestures
- **Robotic Teleoperation:** Interface with robotic hands for remote manipulation

6.4 Conclusion

This project successfully demonstrated the feasibility of creating an affordable, customizable hand-tracking system using accessible hardware and open-source software. The system effectively bridges physical sensor data with real-time 3D visualization, achieving smooth animation through signal processing and kinematic constraints.

The modular architecture ensures that future developers can build upon this foundation for applications in assistive technology, virtual reality, gaming, and human-computer interaction.

While limitations exist in the current implementation, particularly in the game environment and wireless capabilities, the core objectives were achieved.

As sensor technology advances and computational costs decrease, such systems will become increasingly accessible, ultimately making natural, intuitive hand-based interfaces available to a wider audience. This project serves as a foundational step toward that goal, demonstrating that complex hand tracking is achievable with readily available components and open-source tools.

References

1. X. Zhang, M. Liu, and Y. Zhao, "Design of angle detection system based on MPU6050," in *Proc. Int. Conf. Intelligent Transportation, Big Data & Smart City*, Changsha, China, 2017, pp. 159–162.
2. S. J. Park and S. Y. Lee, "Real-Time Motion Recognition Using an Accelerometer," *Sensors*, vol. 15, no. 1, pp. 135–148, 2015.
3. P. Kumar, J. Verma, and S. Prasad, "Hand data glove: A wearable real-time device for human-computer interaction," *International Journal of Advanced Science and Technology*, vol. 43, pp. 15–26, June 2012.
4. A. Khandual and A. Biswal, "IoT Based Smart Glove for Hand Gesture Recognition," *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 5, no. 2, pp. 384–389, Mar. 2019.
5. M. Nasri et al., "Design and Implementation of a 3D Printed Hand Gesture Recognition System Using Arduino and Flex Sensors," *IEEE Access*, vol. 7, pp. 55371–55378, 2019.
6. T. Baudel and M. Beaudouin-Lafon, "Charade: Remote Control of Objects Using Free-Hand Gestures," *Commun. ACM*, vol. 36, no. 7, pp. 28–35, 1993.
7. S. S. Rautaray and A. Agrawal, "Real time hand gesture recognition system for dynamic applications," *International Journal of UbiComp (IJU)*, vol. 3, no. 1, pp. 21–35, Jan. 2012.
8. S. Chowdhury and A. Haque, "Animatronic hand controller," Technical Report, Dept. of Electrical and Electronic Engineering, Ahsanullah University of Science and Technology, Dhaka, Bangladesh, Dec. 2013.
9. D. J. Sturman and D. Zeltzer, "A survey of glove-based input," *IEEE Computer Graphics and Applications*, vol. 14, no. 1, pp. 30–39, Jan. 1994.
10. J. Huang, "Design of angle detection system based on MPU6050," in *Proc. 7th Int. Conf. Education, Management, Information and Computer Science (ICEMC 2017)*, *Advances in Computer Science Research*, vol. 73, pp. 1–3, 2017.